

ByteBridge: Diagnosing the Limits of Byte Input Adapters for Frozen Tokenizer-Based Language Models

Anonymous Authors

Paper under double-anonymous review

Abstract. Subword tokenizers are efficient input interfaces for common text, but they can be brittle on typos, Unicode text, code, multilingual text, and unusual strings. Byte-level models avoid fixed vocabularies when trained from scratch, but many useful language models already exist with fixed tokenizers. We ask whether such a model can be kept frozen and given an input-side byte interface through only a small trainable adapter. ByteBridge tests this question with a paired protocol: input context is derived from bytes, target tokens remain native, and the LM backbone and head stay frozen. On Qwen2.5, a ladder from input-layer byte patches to KV-prefix memory improves typo-noise robustness but does not match native tokenization on clean or non-typo stress inputs. On TinyLlama, the deeper KV interface is stronger and can outperform native tokenization on clean, typo, Unicode, and multilingual buckets when multilingual examples are included in training, while tokenizer-stress strings remain difficult. Together, these results identify a practical limit of byte-side retrofitting: frozen tokenizer-based models do not merely consume embedding vectors, but also depend on token units, boundaries, and positions induced during pretraining.

Keywords: Tokenization · Byte-level modeling · Frozen language models · Adapter tuning · Diagnostic study

1 Introduction

Subword tokenizers are the front door of most language models. They turn raw text into the discrete units that the model was trained to read. This works well for common text, but it can be brittle. A small typo may change the segmentation. Rare Unicode characters may become many tokens. Code, file paths, long identifiers, and multilingual text may be split in ways that look different from clean English.

Byte-level modeling avoids out-of-vocabulary inputs because every string can be written as bytes. Recent byte, character, and adaptive-tokenization models improve coverage, robustness, multilingual segmentation, and efficiency when trained for this format from the beginning [1,2,6,11,16,17,19,20]. But many useful language models are not byte models. They were trained with a fixed tokenizer. A

tempting shortcut is to keep such a model frozen and train only a small adapter that turns input bytes into continuous vectors the model can use.

This paper tests that shortcut. We ask: *can a small byte adapter give useful input context to a frozen tokenizer-based language model, without badly hurting clean-text performance?* We study this with ByteBridge, primarily on Qwen2.5 causal language models [13], with a selected TinyLlama replication [21]. The language-model backbone, input embedding table, and LM head are always frozen. Only byte-side adapters are trained. The output side still uses each model’s native tokenizer, so this is not a fully tokenizer-free language model. It is a test of whether the *input side* of an existing tokenizer-based model can be replaced or augmented by bytes.

There are two plausible outcomes. The optimistic view is that a byte adapter can map UTF-8 bytes into vectors that a frozen model can use, especially if those vectors are aligned to token embeddings or inserted through prompt- or prefix-tuning interfaces [8,9]. The pessimistic view is that a tokenizer-based model learns tokenizer-induced structure beyond token embeddings: units, boundaries, positions, and attention patterns. A small adapter may not recover all of that structure.

We evaluate these possibilities with a diagnostic ladder. We first test input-side byte interfaces at the input layer, then alignment and reconstruction objectives, and finally deeper byte-derived memory through soft-prefix and per-layer KV-prefix injection. The same evaluation suite is reused throughout: clean English, typo noise, Unicode stress, multilingual text, code, and tokenizer-stress strings.

The main result is mixed. On Qwen2.5, byte adapters consistently beat the native tokenizer baseline on typo-noise prompts, but every byte adapter is worse on clean text, and the native tokenizer remains better on Unicode, code, and tokenizer-stress inputs. Deeper byte-derived memory helps: all-layer KV-prefix gives the best Qwen2.5-0.5B clean retention, $1.61\times$ the tokenizer baseline, but still misses our $1.5\times$ diagnostic threshold. On TinyLlama-1.1B-Chat, the same all-layer KV-prefix interface is much stronger: over three seeds it beats the tokenizer baseline on clean, typo-noise, and Unicode buckets. A multilingual training ablation then shows that the severe held-out multilingual failure is not intrinsic to byte input, but depends on coverage and model family. This cross-family contrast is the central finding: byte adapters can help, but they are not a general drop-in replacement for the tokenizer-induced structure that the frozen model was trained to use.

This paper makes four contributions. 1) We build a paired benchmark and scoring protocol for byte-side input adapters with fixed target-token evaluation. 2) We compare input-layer byte patches, alignment and reconstruction objectives, soft prefixes, and all-layer KV-prefix memory, showing that deeper injection helps but does not close the Qwen2.5 clean-retention gap. 3) We add preprocessing controls, fragmentation analysis, cross-model replications, and a train-only multilingual ablation to separate cleanup, fragmentation, model-family, and coverage effects. 4) We report a mixed result: Qwen2.5 adapters mainly improve

typo robustness, while TinyLlama KV-prefix adapters are stronger and more coverage-sensitive.

2 Problem Setting

Let M be a pretrained causal language model with tokenizer T , input embedding matrix E , transformer F , and LM head H . In all experiments, E , F , and H are frozen. Given an input string x and target string y , the native tokenizer baseline computes

$$\mathcal{L}_{\text{tok}}(x, y) = - \sum_t \log p_M(y_t \mid T(x), y_{<t}), \quad (1)$$

where y_t are target tokens under the model’s tokenizer. Byte variants replace or augment the conditioning on x with continuous states $A(b(x))$. Here $b(x)$ is the UTF-8 byte sequence and A is the trainable adapter. The target side remains tokenized. This keeps the comparison fair: every system is scored on the same target tokens.

This paired conditional setup is important. Comparisons can be misleading if one system is scored on bytes and another on tokens, or if noisy inputs change the supervised target. We avoid that problem by keeping y fixed. For noisy examples, x is the perturbed string and y is the clean target suffix. Loss is computed only over target tokens, not over byte-prefix or adapter positions.

The main metric is per-bucket target-token loss. We also save perplexity in the experiment artifacts, but focus on loss because stress-bucket perplexities can be very large. To measure clean-text degradation, we use

$$\text{clean retention}(A) = \frac{\mathcal{L}_A(\text{clean})}{\mathcal{L}_{\text{tok}}(\text{clean})}. \quad (2)$$

Before running the diagnostic ladder, we set two practical thresholds: adapters with at most 50% higher clean loss than the tokenizer baseline are treated as diagnostically promising (clean retention ≤ 1.5), and adapters with at most 20% higher clean loss are treated as strong (clean retention ≤ 1.2). These thresholds are diagnostic guardrails, not tuned decision boundaries. They prevent us from calling a byte interface useful when it only wins narrow stress cases by sacrificing clean text.

For key bucket comparisons, we also run paired bootstrap over test examples. Each resample draws examples with replacement within a bucket and recomputes the mean difference in loss, byte adapter minus tokenizer baseline, over 10,000 resamples. A negative difference means the byte adapter has lower loss.

Figure 1 summarizes this paired protocol and the byte-side interfaces tested in the diagnostic ladder.

Table 1 lists the shared model, freezing, training, and split settings used throughout the experiments.

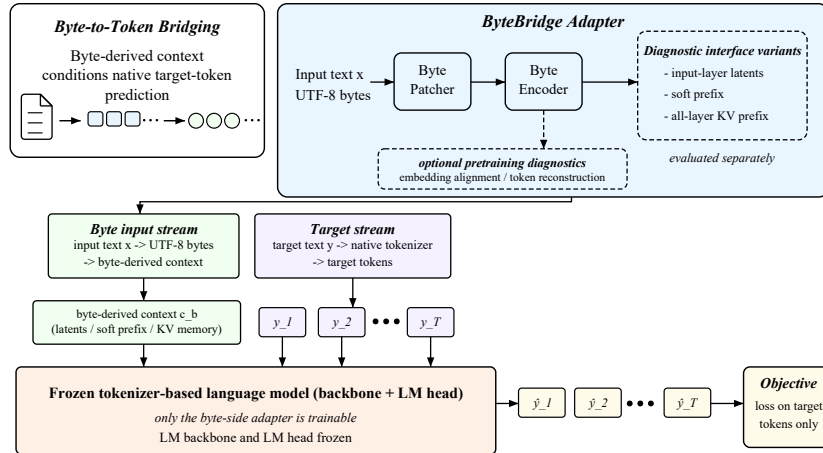


Fig. 1. Overview of the ByteBridge paired protocol.

3 Benchmark and Metrics

We construct a small, reproducible suite that stresses tokenizer-unfriendly inputs while remaining feasible in a single-GPU setting. The train, validation, and test manifests are fixed after construction and reused across all phases; test examples are never used for training. Each record stores its bucket, text fields, source, perturbation metadata, seed, and clean-example identifier where applicable.

Buckets are generated from predefined rules and fixed seeds, not selected after observing model outcomes. Typo variants stay within their original split, train and test share no clean-example identifier, and all released manifests keep source and perturbation metadata. The suite is not a traffic model, but the comparisons are auditable.

The test buckets are:

- **Clean English:** short natural-language, Wikipedia-like, instruction-like, and QA-like snippets.
- **Typo noise:** clean English with random deletion, insertion, substitution, adjacent swaps, repeated characters, whitespace perturbations, and casing noise at light, medium, and heavy levels.
- **Unicode stress:** emoji, accented characters, combining marks, zero-width joiners, full-width characters, mathematical symbols, punctuation variants, and normalized or non-normalized mixtures.
- **Multilingual text:** short Chinese, Japanese, Korean, Arabic, Hindi, Russian, and Latin-script accented examples.
- **Code:** Python, JavaScript, JSON, shell commands, stack traces, regular expressions, SQL, and configuration snippets.
- **Tokenizer-stress strings:** URLs, file paths, package names, UUIDs, base64-like and hexadecimal strings, logs, long identifiers, and mixed separators.

Table 1. Main experimental settings.

Item	Setting
Backbones	Qwen2.5-0.5B for the full ladder; Qwen2.5-1.5B and TinyLlama-1.1B-Chat for selected replications
Frozen parts	LM backbone, input embedding table, and LM head; valid runs require zero trainable LM parameters and zero checksum drift
Trainable parts	Byte-side adapter only: input latents, soft prefix, KV-prefix, or auxiliary reconstruction/distillation heads where applicable
Training	AdamW, learning rate 10^{-3} , 2000 main steps unless noted, single NVIDIA RTX 5090 GPU
Sequence limits	128 prompt bytes and 64 target tokens for paired conditional LM runs
Splits	Fixed Phase 2 train, validation, and test manifests; test examples are never used for training

Table 2. Fixed splits reused across all phases.

Split	examples	avg. input bytes	avg. input toks	avg. target toks
train	5168	121.9	34.1	26.1
validation	1188	99.5	30.9	25.7
test	2028	100.3	30.9	25.8

Table 2 gives the split sizes. The benchmark is synthetic and curated. It is not intended to represent natural text frequencies. Its role is to make failure cases visible under controlled conditions.

The native tokenizer baseline is the frozen Qwen2.5-0.5B evaluated directly on this paired conditional test set. Its clean-English loss is 0.3089. This baseline is intentionally strong because it uses the tokenizer and embedding conventions the model saw during pretraining.

4 ByteBridge Adapter Family

4.1 Input-Layer Byte Latents

The simplest adapter groups bytes into fixed-size patches and maps them into L prompt vectors in the LM hidden dimension. This directly replaces tokenized prompt embeddings through `inputs_embeds`. We train clean-only and noise-augmented variants; the strongest Phase 2 run uses $L = 32$ and 2000 noise-augmented steps.

4.2 Embedding Distillation

One possible failure is that byte latents are far from Qwen token embeddings. We therefore pretrain the adapter with fixed token-span alignment or pooled sequence alignment against frozen token embeddings, then fine-tune it with the same frozen-LM conditional objective.

4.3 Token Reconstruction and Boundary-Aware Patching

Embedding MSE may be too weak if the frozen model needs token-like identity rather than nearby vectors. We map tokenizer tokens to UTF-8 byte spans and pretrain byte/span latents to predict tokenizer token IDs. We test fixed patches, heuristic tokenizer-free boundaries, and oracle tokenizer-derived spans. The oracle mode is a diagnostic upper bound, not a tokenizer-free method; the classifier head is discarded before LM evaluation.

4.4 Soft Prefix Injection

Input replacement may be too strict because it forces byte latents to behave like token embeddings at prompt positions. Soft prefix injection instead compresses input bytes into P learned prefix embeddings, concatenates them before tokenized target embeddings, and computes loss only on target positions:

$$\text{inputs_embeds} = [A_{\text{prefix}}(b(x)); E(T(y_{<t}))]. \quad (3)$$

This keeps native target-side teacher forcing while giving the frozen model byte-derived context.

4.5 KV-Prefix Injection

Finally, we test deeper injection by inserting byte-derived prefix memories into attention key/value inputs. For Qwen’s `DynamicCache`, these memories have shape $[B, H_{kv}, P, d_h]$. We test first-4-layer and all-layer injection; non-injected layers receive zero K/V tensors of the same prefix length for cache consistency. This is a diagnostic compromise, not an optimal prefix-tuning design.

5 Main Results on Qwen2.5-0.5B

5.1 Clean Retention Across the Diagnostic Ladder

Table 3 summarizes the best representative run from each phase. Lower loss is better. A clean retention of $1.61\times$ means 61% higher clean loss than the tokenizer baseline. The pattern is consistent: later diagnostics improve some aspects of earlier interfaces, but none approaches native tokenizer conditioning on clean text. The best input-layer reconstruction run reaches $1.84\times$ clean retention. Soft prefixes improve this to $1.76\times$, and all-layer KV-prefix reaches $1.61\times$. All remain above the $1.5\times$ diagnostic threshold.

Table 4 shows selected bucket losses. The tokenizer baseline remains strongest on clean English, code, tokenizer-stress, Unicode, and multilingual text. The byte adapters are strongest on typo noise. This is the one robust positive signal across phases: when the prompt is corrupted but the target is clean, byte adapters can learn a denoising-style context that degrades less than native tokenized conditioning. Paired bootstrap confirms the sign of the Qwen2.5-0.5B KV-prefix typo

Table 3. Representative Qwen2.5-0.5B runs across the diagnostic ladder.

Stage	representative run	clean loss	clean retention wins
Tokenizer	native Qwen tokenizer	0.3089	1.00 –
Phase 2	fixed input, noise, $L = 32$	0.6770	2.19 typo L/M/H
Phase 3	distill then LM, $L = 32$	0.6863	2.22 typo L/M/H
Phase 4	fixed recon then LM	0.5669	1.84 typo L/M/H
Phase 5	soft prefix, $P = 64$	0.5441	1.76 typo L/M/H
Phase 6	KV prefix, $P = 32$, all layers	0.4985	1.61 typo L/M/H

Table 4. Selected Qwen2.5-0.5B test losses.

Bucket	tokenizer baseline	Phase 2 best	Phase 4 fixed recon	soft prefix p64	KV prefix p32 all
clean English	0.3089	0.6770	0.5669	0.5441	0.4985
code	0.3876	2.4261	2.0058	0.7878	1.3462
tokenizer stress	0.4806	3.1706	2.8533	1.5118	1.5969
Unicode stress	0.5614	3.8226	2.3900	0.9630	0.9809
typo light	1.3838	0.7454	0.6416	0.5817	0.5167
typo medium	1.8428	0.7786	0.6626	0.6097	0.5298
typo heavy	2.8353	0.8203	0.7749	0.6686	0.5986
multilingual zh	0.6085	4.5419	4.7134	4.9432	11.1217
multilingual ar	0.3806	3.9973	4.4678	5.4298	10.7503

gains and of its clean, Unicode, tokenizer-stress, and multilingual losses: the 95% confidence intervals for byte-minus-tokenizer loss do not cross zero.

Deeper injection helps non-typo stress relative to earlier byte variants. Soft prefix p64 reduces code loss from 2.0058 under fixed reconstruction to 0.7878, tokenizer-stress from 2.8533 to 1.5118, and Unicode from 2.3900 to 0.9630. However, these losses remain worse than tokenizer baseline losses of 0.3876, 0.4806, and 0.5614. All-layer KV-prefix gives the best clean loss, but is worse than soft prefix p64 on code and tokenizer-stress, and multilingual losses remain poor.

5.2 What Each Diagnostic Tells Us

Fixed Byte Patches Are Insufficient. The Phase 2 input-layer adapter confirms engineering feasibility: gradients flow through `inputs_embeds`, LM trainable parameters remain zero, and checksum drift is zero. But its best clean retention is only $2.19\times$, with severe degradation on Unicode, code, multilingual, and tokenizer-stress buckets.

Embedding Alignment Is Not Enough. Projection alignment works as an auxiliary objective: distillation loss drops from 0.9583 to 0.0637 for $L = 32$ and from 0.9654 to 0.0752 for $L = 64$. Downstream LM evaluation does not improve: the $L = 32$ distill-then-LM run has $2.22\times$ clean retention. Matching token embeddings geometrically is therefore not enough.

Table 5. Deterministic preprocessing controls on Qwen2.5-0.5B.

Mode	changed frac.	clean	typo L	typo M	typo H
raw tokenizer	0.000	0.3089	1.3838	1.8428	2.8353
NFC	0.015	0.3089	1.3838	1.8428	2.8353
whitespace cleanup	0.112	0.3089	1.3518	1.8257	2.8137
repeat squeeze	0.039	0.3089	1.3835	1.8429	2.8189
casefold	0.693	0.6028	1.0721	1.4980	2.4380
typo cleanup	0.709	0.6028	1.0748	1.5098	2.4155
oracle clean typo	0.296	0.3089	0.3039	0.3039	0.3039
soft prefix p64	–	0.5441	0.5817	0.6097	0.6686
KV prefix p32 all	–	0.4985	0.5167	0.5298	0.5986

Token Identity Is Learnable, but Not Sufficient. Token reconstruction is stronger than embedding alignment. Fixed reconstruction reaches validation clean top-1 accuracy 0.8669, and oracle-boundary reconstruction reaches 0.9945. Yet downstream clean retention remains poor: fixed reconstruction improves to $1.84\times$, while oracle-boundary reconstruction is $2.20\times$. Boundary and identity errors are real bottlenecks, but not the only bottlenecks.

Deeper Injection Helps, but Not Enough. Soft prefix and KV-prefix variants test whether the frozen model needs byte information as attention context rather than replacement input embeddings. The answer is partially yes: soft prefix p64 improves code, Unicode, and tokenizer-stress losses over input-layer variants, and all-layer KV-prefix gives the best clean retention, $1.61\times$. But neither reaches $1.5\times$ clean retention, and neither beats the tokenizer baseline on non-typo stress buckets.

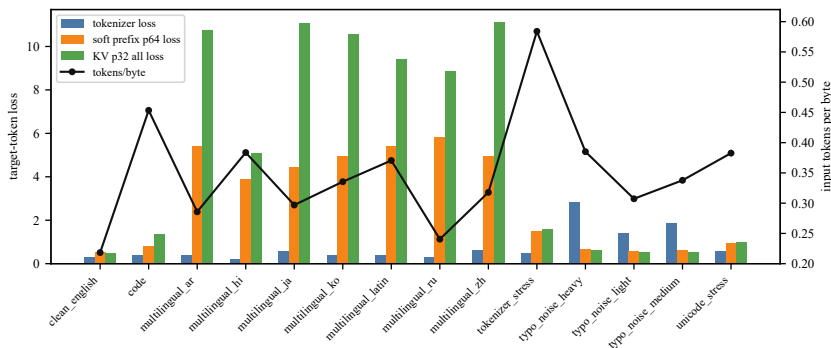
6 Is the Typo Gain Just Preprocessing?

A natural objection is that the typo-noise gains may be explained by simple input cleanup. We therefore evaluate deterministic preprocessing baselines on the native tokenizer path. Each transform is applied only to the input side; the target text and target-token scoring stay unchanged. Table 5 summarizes the main controls.

Conservative cleanup, such as Unicode NFC normalization, whitespace cleanup, and repeated-character squeezing, barely changes the typo buckets. Stronger cleanup, such as case folding and our combined typo cleanup, improves typo loss but roughly doubles clean loss from 0.3089 to 0.6028. Byte adapters remain much better on typo loss than these deterministic transforms. The oracle row shows that the paired task can be easy if the clean prompt is known, but this is an upper bound, not a deployable baseline. The typo signal is therefore real, but narrow: it does not imply broad tokenizer-free superiority.

Table 6. Selected fragmentation correlations over Qwen2.5-0.5B test samples.

Predictor	outcome	Pearson	Spearman
input token length	tokenizer loss	0.669	0.214
tokens per byte	tokenizer loss	0.227	0.334
bytes per token	tokenizer loss	-0.319	-0.334
input token length	soft prefix p64 gain	0.276	0.459
input token length	KV-prefix gain	0.349	0.477

**Fig. 2.** Bucket-level loss and fragmentation view.

7 Mechanism Analysis

7.1 Fragmentation Correlates with Loss, but Does Not Explain Everything

Tokenizer-unfriendly strings often produce longer token sequences. We measure several fragmentation proxies on the fixed test set: input byte length, input token length, tokens per byte, and bytes per token. We then correlate these with tokenizer loss and with byte-adapter gain over the tokenizer baseline.

Table 6 gives selected correlations. Input token length has a strong Pearson correlation with tokenizer loss (0.669) but a much weaker Spearman correlation (0.214), meaning length is useful but not a complete ranking signal. Tokens per byte has a smaller Pearson correlation (0.227) and a moderate Spearman correlation (0.334). Byte adapters gain more on longer tokenized inputs, with Spearman correlations of 0.459 for soft prefix p64 and 0.477 for KV-prefix p32 all-layer. Still, high fragmentation is not enough to predict success. Without multilingual adapter training, multilingual buckets are highly problematic even though byte input should, in principle, avoid vocabulary coverage issues. Figure 2 summarizes this bucket-level pattern.

7.2 Interpretation

The diagnostics narrow the failure mode. Embedding alignment, oracle token boundaries, and deeper prefix injection each help only partially. If the issue were

only embedding geometry, boundary segmentation, or shallow input injection, one intervention should have closed the clean-retention gap. None did.

Our interpretation is that frozen tokenizer-based language models depend on several parts of tokenizer-induced structure at once. Native token embeddings carry local text content, but the model also expects familiar token units, boundary lengths, position steps, and attention patterns. A small input-side byte interface trained after the fact can learn useful robust cues, especially for typo-noise denoising, but it does not recreate the full input interface.

The multilingual ablation in Section 8.1 refines this point. When multilingual text is absent from adapter training, held-out multilingual buckets fail catastrophically. Adding multilingual training examples largely removes that failure for TinyLlama, but Qwen still pays a large clean-retention cost. The issue is therefore not byte representation alone; it is coverage, capacity, and compatibility with the frozen model’s tokenizer-induced interface.

8 Cross-Model Replication

To test whether the Qwen result is model-family specific, we run selected replications on two additional frozen models. This is not a full rerun of the entire diagnostic ladder. It is a focused check of the best later-stage interfaces: soft prefix p64 and all-layer KV-prefix p32.

The first replication uses Qwen2.5-1.5B. This keeps the tokenizer family and cache API close to the primary model, reducing engineering confounds while testing a larger frozen Qwen2.5 model. The native Qwen2.5-1.5B tokenizer clean loss is 0.3274. Soft prefix p64 improves with longer training, from $6.22\times$ clean retention at 2000 steps to $2.73\times$ at 8000 steps, but remains far above the $1.5\times$ threshold. All-layer KV-prefix is the best selected Qwen2.5-1.5B run, with $2.52\times$ clean retention. It beats the tokenizer baseline only on medium and heavy typo noise.

The second replication uses TinyLlama-1.1B-Chat, which changes the tokenizer and model family. Here the result is different. The native tokenizer clean loss is 0.6026. Soft prefix p64 reaches $1.26\times$ clean retention and beats the tokenizer baseline on all three typo buckets. All-layer KV-prefix p32 is much stronger: across three seeds, its mean clean retention is $0.63\times$, and the seed values are 0.69, 0.66, and 0.55. The three-seed mean beats the tokenizer baseline on clean English, all typo buckets, and Unicode stress, while losing on tokenizer-stress and on multilingual buckets that were absent from adapter training. Paired bootstrap confirms these signs: 95% intervals do not cross zero for the clean, typo, and Unicode gains, or for the tokenizer-stress and held-out multilingual losses. A target-only sanity baseline has clean loss 5.4014, and a learned constant KV-prefix that never reads input bytes has clean loss 0.8907. These controls show that target-side prediction and generic soft prompting do not explain the full TinyLlama KV result. Table 7 summarizes the selected cross-model replications. In the bucket table, positive byte gain means lower loss than the tokenizer baseline; negative gain means degradation.

Table 7. Selected cross-model replication.

Model	run	clean loss	retention	steps	wins
Qwen2.5-1.5B	tokenizer	0.3274	1.00	0	–
Qwen2.5-1.5B	prefix p64	0.8927	2.73	8000	typo H
Qwen2.5-1.5B	KV p32 all	0.8245	2.52	2000	typo M/H
TinyLlama-1.1B-Chat	target-only	5.4014	8.96	0	none
TinyLlama-1.1B-Chat	tokenizer	0.6026	1.00	0	–
TinyLlama-1.1B-Chat	prefix p64	0.7609	1.26	2000	typo L/M/H
TinyLlama-1.1B-Chat	constant KV	0.8907	1.48	2000	typo L/M/H
TinyLlama-1.1B-Chat	KV p32 all	0.3811	0.63	2000	clean; L/M/H; Uni-code

Table 8. TinyLlama-1.1B-Chat bucket-level results without multilingual adapter training.

Bucket	tokenizer	constant KV	byte KV	byte gain
clean English	0.6026	0.8907	0.3811	36.8%
typo light	1.4080	0.8869	0.3823	72.8%
typo medium	1.7133	0.8870	0.3930	77.1%
typo heavy	2.5426	0.8870	0.4313	83.0%
Unicode stress	0.7818	1.9301	0.6620	15.3%
code	0.6087	1.5146	0.7778	-27.8%
tokenizer stress	0.6057	2.1189	1.2281	-102.8%
multilingual zh	0.5049	4.3321	10.7154	-2022.3%
multilingual ar	0.4396	4.4745	11.1403	-2434.3%

8.1 Interpreting the TinyLlama-Qwen Gap

The TinyLlama result changes the interpretation, but not the caution. Qwen shows that byte adapters are not reliable drop-in replacements: even all-layer KV-prefix misses the clean-retention threshold and never beats the tokenizer outside typo noise. TinyLlama shows the other side of the boundary: the failure is not universal, and some frozen models can use byte-derived KV memory better than their tokenized prompt.

We do not identify a single cause for the TinyLlama-Qwen gap. Two clues are useful but incomplete. TinyLlama’s native tokenizer baseline is weaker on this benchmark, with clean loss 0.6026 rather than Qwen2.5-0.5B’s 0.3089. It also tokenizes the same inputs more densely, with higher input tokens per byte on clean text (0.241 vs. 0.218), Unicode (0.493 vs. 0.383), code (0.533 vs. 0.453), tokenizer-stress strings (0.700 vs. 0.584), Chinese (0.550 vs. 0.318), and Arabic (0.547 vs. 0.286). This may leave more room for byte-conditioned KV memory, but fragmentation is not sufficient: TinyLlama’s multilingual inputs are also highly fragmented, yet byte KV fails when these scripts are absent from adapter training.

Table 9. Multilingual training ablation for all-layer KV-prefix adapters.

Model	train mix	clean retention	avg. multi loss	multi wins
Qwen2.5-0.5B	no multi	1.61	9.55	0/7
Qwen2.5-0.5B	+multi	2.24	0.49	2/7
TinyLlama-1.1B-Chat	no multi	0.69	10.53	0/7
TinyLlama-1.1B-Chat	+multi	0.57	0.33	7/7

Training curves rule out a stalled optimizer: Qwen2.5-0.5B drops from training loss 8.61 to 0.25, and the three TinyLlama seeds drop from 10.36/11.28/10.76 to 0.21/0.30/0.22. The gap appears in validation and test behavior. TinyLlama reaches validation clean losses of 0.30–0.39, while Qwen reaches 0.468. A constant TinyLlama KV-prefix that does not read input bytes has test clean loss 0.8907, so the TinyLlama gain is not just target-side prediction, generic soft prompting, or optimizer success.

We also compare each learned byte KV cache with the native cache produced by tokenized prompts. This does not support the simple idea that TinyLlama works because its byte KV tensors are globally closer to native KV tensors. Qwen’s byte-to-native key/value RMS ratios are 0.69/3.12, while TinyLlama’s are 1.61/19.26. TinyLlama has only slightly higher mean-vector cosine (0.071/0.039 for key/value, versus Qwen’s 0.021/−0.032). The result points instead to a sensitivity difference: TinyLlama can use some high-magnitude byte-derived prefix memory, even though that memory is not a faithful copy of its native prompt cache.

The multilingual failures in Table 8 are held-out failures: the adapters were trained on clean English, typo noise, Unicode, code, and tokenizer-stress buckets, but not on multilingual buckets. To test whether this is an intrinsic byte-interface failure or a coverage failure, we add a train-only multilingual supplement of 1120 examples, 160 for each of seven language groups, and keep the validation and test manifests unchanged. Table 9 summarizes the result.

Multilingual failure is not proof that byte inputs cannot represent non-Latin text. With multilingual training, TinyLlama beats the tokenizer baseline on all seven multilingual buckets and improves clean retention from $0.69\times$ to $0.57\times$. Qwen also reduces average multilingual loss from 9.55 to 0.49, but clean retention worsens from $1.61\times$ to $2.24\times$ and only two multilingual buckets beat the tokenizer. Thus multilingual behavior is coverage-sensitive, but the Qwen trade-off remains: the byte adapter can learn script-specific cues, yet doing so does not make it a reliable replacement for native tokenization.

9 Related Work

Subword Tokenization. Subword tokenizers such as byte-pair encoding, WordPiece, and SentencePiece are widely used because they balance vocabulary size and open-vocabulary coverage [3,7,15]. Prior work has shown that tokenization choices affect robustness, fairness, multilingual efficiency, language variation,

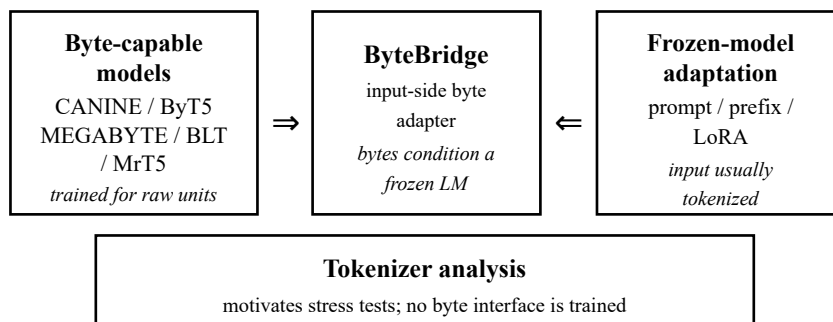


Fig. 3. Position of ByteBridge relative to nearby work.

byte-level probabilities, and token-level vulnerabilities [1,5,10,12,14,18]. ByteBridge focuses on a complementary question: whether an already-tokenized language model can be retrofitted with a byte-based input side while keeping the model frozen.

Byte and Character-Level Language Models. Byte and character models show that raw input units can work when the model is trained for them [2,6,11,16,17,19,20]. Figure 3 summarizes why these are not direct baselines for our frozen-adapter setting.

Parameter-Efficient Frozen-Model Adaptation. Prompt tuning, prefix tuning, and LoRA show that frozen or mostly frozen LMs can be steered by small trainable modules [4,8,9]. Our task is different: the trainable module must encode the raw input string itself from bytes, not only provide a task prompt. This is why we include a constant-prefix control.

Diagnostic Negative and Mixed Results. ByteBridge is a diagnostic study of when a tempting shortcut fails or works only under narrow conditions. Embedding alignment, oracle boundary reconstruction, and deeper prefix injection are each insufficient for reliable Qwen2.5 byte adapters. TinyLlama adds the complementary lesson that the failure is not universal.

10 Limitations

This study is scoped to Qwen2.5 and TinyLlama, small adapters, a single-GPU environment, and a synthetic or curated stress benchmark. The suite is for controlled diagnosis, not for estimating real-world frequency or user impact; real text may contain different noise patterns, longer contexts, mixed tasks, and domain-specific conventions. The typo gains should therefore be read as evidence that byte adapters can use noisy byte cues under this paired benchmark, not as a deployment claim about all user typos.

We do not claim that all byte interfaces for all frozen LMs must fail, or that TinyLlama’s success transfers to other model families. We also do not eliminate the target-side tokenizer: supervision and output scoring use each model’s native target tokens for fair comparison. The study has not tested many tokenizer families, substantially larger adapters, longer schedules, contrastive pretraining, hard negatives, or partial LM finetuning. Oracle-boundary reconstruction uses tokenizer-derived boundaries and is a diagnostic upper bound; first- N KV-prefix variants use zero K/V tensors for non-injected layers to maintain cache length consistency, which may not be optimal.

11 Conclusion

ByteBridge asks whether a small input-side byte interface can replace or augment native tokenized input for a frozen tokenizer-based language model. The answer is mixed. Under Qwen2.5, byte adapters produce a consistent typo-noise robustness signal and deeper byte-derived memory improves several stress categories, but clean retention remains too poor and native tokenization continues to dominate non-typo stress buckets. Under TinyLlama-1.1B-Chat, all-layer KV-prefix injection is much stronger and beats native tokenization on clean, typo, and Unicode buckets; with multilingual training, it also beats the tokenizer on multilingual buckets, but tokenizer-stress remains difficult. Deterministic preprocessing controls do not explain away the typo signal, and fragmentation analysis shows only a partial mechanism. The main lesson is that tokenization induces input structure that a byte adapter must recover or replace.

Reproducibility statement. All experiments save configuration files, environment metadata, training metrics, summaries, checkpoints, and per-bucket evaluation outputs. The Phase 2 train, validation, and test manifests are reused unchanged across all later diagnostic and control experiments. In all reported LM runs, the frozen model trainable parameter count is zero and the frozen checksum delta is zero.

References

1. Ahia, O., Kumar, S., Gonen, H., Hofmann, V., Limisiewicz, T., Tsvetkov, Y., Smith, N.A.: MAGNET: Improving the multilingual fairness of language models with adaptive gradient-based tokenization. In: *Advances in Neural Information Processing Systems*. vol. 37 (2024)
2. Clark, J.H., Garrette, D., Turc, I., Wieting, J.: CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics* **10**, 73–91 (2022)
3. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. pp. 4171–4186 (2019)

4. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: LoRA: Low-rank adaptation of large language models. In: International Conference on Learning Representations (2022)
5. Jang, E., Lee, K., Chung, J.W., Park, K., Shin, S.: Improbable bigrams expose vulnerabilities of incomplete tokens in byte-level tokenizers. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. pp. 18209–18216. Association for Computational Linguistics, Suzhou, China (2025)
6. Kallini, J., Murty, S., Manning, C.D., Potts, C., Csordás, R.: MrT5: Dynamic token merging for efficient byte-level language models. In: The Thirteenth International Conference on Learning Representations (2025)
7. Kudo, T., Richardson, J.: SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (2018)
8. Lester, B., Al-Rfou, R., Constant, N.: The power of scale for parameter-efficient prompt tuning. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (2021)
9. Li, X.L., Liang, P.: Prefix-tuning: Optimizing continuous prompts for generation. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (2021)
10. Mielke, S.J., Alyafeai, Z., Salesky, E., Raffel, C., Dey, M., Gallé, M., Raja, A., Si, C., Lee, W.Y., Sagot, B., Tan, S.: Between words and characters: A brief history of open-vocabulary modeling and tokenization in NLP. CoRR **abs/2112.10508** (2021)
11. Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., Zhou, C., Yu, L., Weston, J.E., Zettlemoyer, L., Ghosh, G., Lewis, M., Holtzman, A., Iyer, S.: Byte latent transformer: Patches scale better than tokens. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 9238–9258 (2025)
12. Phan, B., Amos, B., Gat, I., Havasi, M., Muckley, M.J., Ullrich, K.: Exact byte-level probabilities from tokenized language models for FIM-tasks and model ensembles. In: The Thirteenth International Conference on Learning Representations (2025)
13. Qwen Team: Qwen2.5 technical report. arXiv preprint arXiv:2412.15115 (2024)
14. Rust, P., Pfeiffer, J., Vulić, I., Ruder, S., Gurevych, I.: How good is your tokenizer? on the monolingual performance of multilingual language models. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (2021)
15. Sennrich, R., Haddow, B., Birch, A.: Neural machine translation of rare words with subword units. In: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (2016)
16. Slagle, K.: SpaceByte: Towards deleting tokenization from large language modeling. In: Advances in Neural Information Processing Systems. vol. 37 (2024)
17. Tay, Y., Tran, V.Q., Ruder, S., Gupta, J., Chung, H.W., Bahri, D., Qin, Z., Baumgartner, S., Yu, C., Metzler, D.: Charformer: Fast character transformers via gradient-based subword tokenization. In: International Conference on Learning Representations (2022)
18. Wegmann, A., Nguyen, D., Jurgens, D.: Tokenization is sensitive to language variation. In: Findings of the Association for Computational Linguistics: ACL 2025. pp. 10958–10983. Association for Computational Linguistics, Vienna, Austria (2025)
19. Xue, L., Barua, A., Constant, N., Al-Rfou, R., Narang, S., Kale, M., Roberts, A., Raffel, C.: ByT5: Towards a token-free future with pre-trained byte-to-byte models. Transactions of the Association for Computational Linguistics **10**, 291–306 (2022)

20. Yu, L., Simig, D., Flaherty, C., Aghajanyan, A., Zettlemoyer, L., Lewis, M.: MEGABYTE: Predicting million-byte sequences with multiscale transformers. In: Advances in Neural Information Processing Systems (2023)
21. Zhang, P., Zeng, G., Wang, T., Lu, W.: TinyLlama: An open-source small language model. CoRR **abs/2401.02385** (2024)